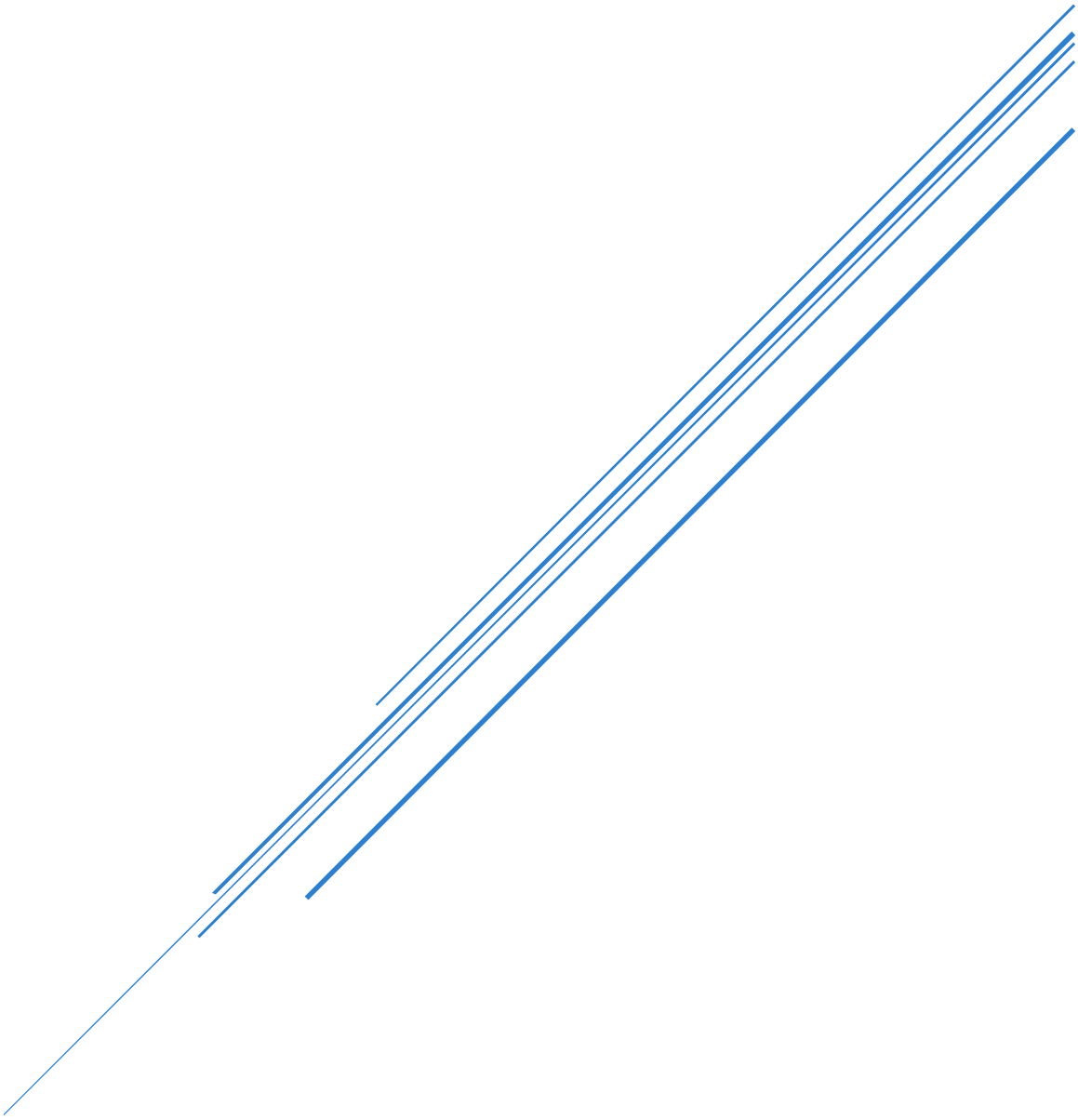
AUTHENTICATION AND ACCESS CONTROL

Assignment - 3



1. **Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.**

➢

```
<!DOCTYPE html>
<html>
<head>
  <title>Zomato Admin Login</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f5f5f5;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }

    .login-box {
      background: white;
      padding: 30px;
      width: 300px;
      border-radius: 10px;
      text-align: center;
      box-shadow: 0 0 10px #ccc;
    }
```

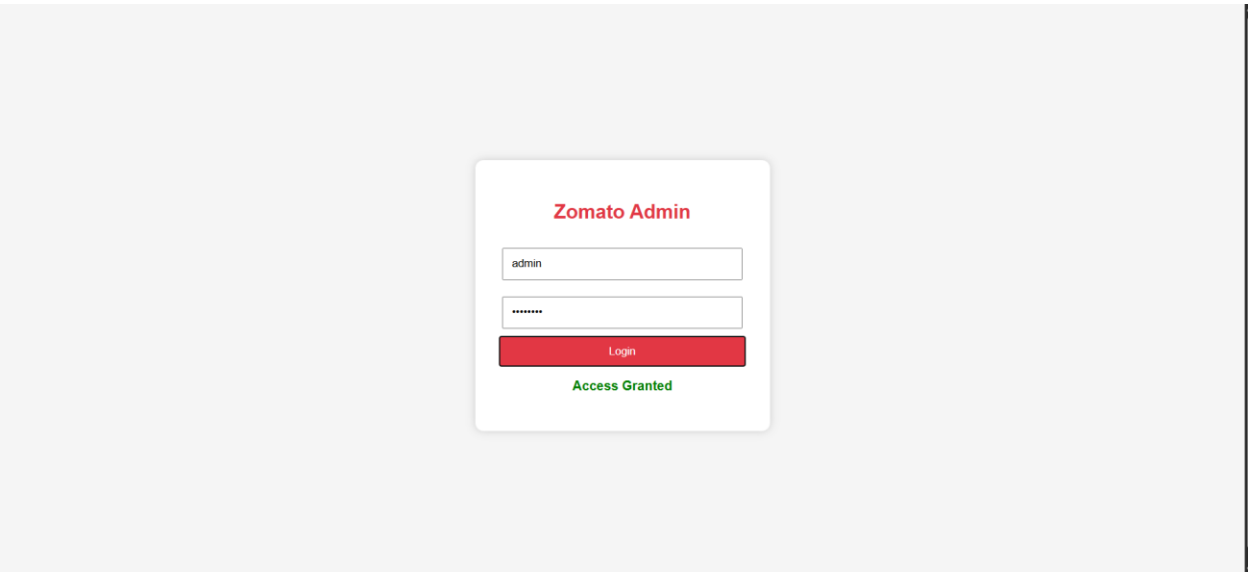
```
    h2 {
      color: #e23744;
    }

    input {
      width: 90%;
      padding: 10px;
      margin: 10px 0;
    }

    button {
      width: 100%;
      padding: 10px;
      background: #e23744;
      color: white;
      border: none;
      cursor: pointer;
    }
    #message {
      margin-top: 15px;
      font-weight: bold;
    }
  </style>
</head>
<body>
```

```
<div class="login-box">
  <h2>Zomato Admin</h2>
  <input type="text" id="username"
    placeholder="Enter Username"
  <input type="password" id="password"
    placeholder="Enter Password">
  <button onclick="login()">Login</button>
  <p id="message"></p>
</div>
<script>
  function login() {
    let username = document.getElementById("username").value;
    let password = document.getElementById("password").value;
    // Hardcoded admin credentials
    let adminUsername = "admin";
    let adminPassword = "admin123";
    if (username === adminUsername &&
      password === adminPassword) {
      document.getElementById("message").innerHTML =
        "Access Granted";
      document.getElementById("message").style.color = "green";
    } else {
      document.getElementById("message").innerHTML =
        "Access Denied";
      document.getElementById("message").style.color = "red";
    }
```

```
    }
  </script>
</body>
</html>
```



2. **Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.**
   Hint: Use a dictionary to map roles to allowed resources.

➢

```
def check_access(user_role, resource):

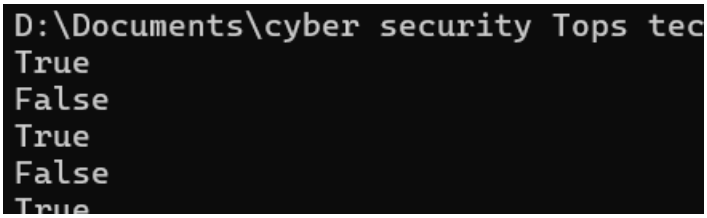
    # Dictionary of roles and their allowed resources
    permissions = {
        "customer": ["menu_edit", "order_history"],
        "restaurant_owner": ["menu_edit", "order_history"],
        "admin": ["menu_edit", "order_history", "admin_dashboard"]
    }

    # Check access permission
    if user_role in permissions:
        if resource in permissions[user_role]:
            return True

    return False


# Testing the function
print(check_access("customer", "order_history"))
print(check_access("customer", "admin_dashboard"))
```

```
print(check_access("restaurant_owner", "menu_edit"))
print(check_access("restaurant_owner", "admin_dashboard"))
print(check_access("admin", "admin_dashboard"))
```



3. **Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.**

➢

```
import random

def login():

    # Hardcoded login credentials
    correct_password = "paytm123"

    password = input("Enter your password: ")

    # Step 1: Check password
    if password == correct_password:

        print("Password Correct!")

        # Step 2: Generate 6-digit OTP
        otp = random.randint(100000, 999999)

        print("Your OTP is:", otp)

        # Step 3: Ask user to enter OTP
        user_otp = input("Enter your OTP: ")
```

```
    # Step 4: Verify OTP
    if user_otp == str(otp):
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed")

else:
    print("Incorrect Password. Access Denied")


# Run login function
login()
```

```
D:\Documents\cyber security Tops technology\Networking\Assig
Enter your password: paytm123
Password Correct!
Your OTP is: 888436
Enter your OTP: 888436
Login Successful
```

4. **Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.**
   Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

➢

```
const users = [
  {
    username: "Rahul",
    isPremium: true,
    lastLogin: "2024-06-08"
  },
  {
    username: "Amit",
    isPremium: false,
    lastLogin: "2024-06-09"
  },
  {
    username: "Priya",
    isPremium: true,
    lastLogin: "2024-06-01"
  },
  {
    username: "Neha",
    isPremium: true,
```

```
    lastLogin: "2024-06-05"
  },
  {
    username: "Raj",
    isPremium: true,
    lastLogin: "2024-06-10"
  }
];

function getPremiumUsers(users) {
  const today = new Date("2024-06-10");
  const sevenDaysAgo = new Date("2024-06-03");

  return users.filter(user => {
    const loginDate = new Date(user.lastLogin);
    return user.isPremium &&
        loginDate >= sevenDaysAgo &&
        loginDate <= today;
  });
}
const result = getPremiumUsers(users);
console.log(result);
```

```
D:\Documents\cyber security Tops technology\Networking\Assignment>node prem
[
  { username: 'Rahul', isPremium: true, lastLogin: '2024-06-08' },
  { username: 'Neha', isPremium: true, lastLogin: '2024-06-05' },
  { username: 'Raj', isPremium: true, lastLogin: '2024-06-10' }
]
```

5. **Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.**

➢

No.	Common Access Control Mistake	Fix / Prevention Tip
1.	Broken Authentication: Weak passwords or improper login verification.	Use strong password policies and multi-factor authentication (MFA).
2.	Unauthorized Data Access: Users can view another user's personal information or orders.	Verify user permissions before allowing access to private data.
3.	Privilege Escalation: Normal users can access admin features.	Apply role-based access control (RBAC) and restrict admin permissions.
4.	Insecure Direct Object Reference (IDOR): Users can access another user's order by changing the order ID in the URL.	Check resource ownership on the server before returning data.
5.	Missing Access Control Checks: Developers forget to check permissions on certain pages or API endpoints.	Implement server-side authorization checks for every protected request.